Senior Design Autobot Project — Navilott: Part One

Mike Orduna
Ana Carlos
Ignacio Rubio

October 17, 2026

Contents

1 Abstract **1**

2 Overview **1**
2.1 Purpose of This Report 1
2.2 The Team 1
2.3 Development Process 1
2.4 How to Read This Report 1

3 Problem **1**
3.1 Objective 1
3.2 Constraints 1
3.3 The Course 2
3.4 Requirements 2

I Design **3**

4 Architecture **3**
4.1 Structure of an Autonomous Robot 3
4.2 Single-Board Design 4
4.3 Additional Hardware 6
4.4 Software Language 8
4.5 Background Threads 9
4.6 Linux Scheduler 12
4.7 Software Architecture 13

5 Capture **13**
5.1 Camera Path 13
5.2 Design Decisions 14
5.3 Failure Handling 14
5.4 Open Questions 14

6 Perception **14**
6.1 Approach 14
6.2 Stage Order 14
6.3 Lane Boundaries and Offset 14
6.4 Stop Lines 14
6.5 Traffic Lights 14
6.6 Stop Signs 15
6.7 Debugging Perception 15

7 Estimation **15**
7.1 Role . . . 15
7.2 Lane Filter . . . 15
7.3 Heading Tracker . . . 15
7.4 Voting . . . 15
7.5 The Packet . . . 15

8 Navigation **15**
8.1 Scope and Attribution . . . 15
8.2 The Contract . . . 16
8.3 Driving Rules . . . 16
8.4 Measured Robot Response . . . 16
8.5 Intersections and Route . . . 16

II Evidence 17

9 Testing **17**
9.1 Scope . . . 17
9.2 Testing Without the Robot . . . 17
9.3 The Suite . . . 17
9.4 What Testing Proves . . . 17

10 Diagnostics **17**
10.1 Why Instrument the Pi . . . 17
10.2 What Is Recorded . . . 17
10.3 Observer Effect . . . 18
10.4 What It Showed . . . 18

11 Verification **18**
11.1 What Counts as Verified . . . 18
11.2 Routines . . . 18
11.3 Checks by Requirement . . . 18
11.4 Whole-Robot Checks . . . 18

12 Results **18**
12.1 Status by Requirement . . . 18
12.2 Routine Results . . . 19
12.3 Course Runs . . . 19
12.4 Measured Robot Response . . . 19

III Assessment 20

13 Limitations **20**
13.1 Requirements Not Met 20
13.2 Requirements Not Yet Shown 20
13.3 Departures from the PRD 20
13.4 Limits of the Design 20

14 Discussion **20**
14.1 What Worked 20
14.2 What Didn't 20
14.3 What I Would Do Differently 20
14.4 What Remains 20

15 Conclusion **21**

IV Supplements **22**

A Hardware **22**

B Parameters **22**
B.1 Capture 22
B.2 Perception 22
B.3 Estimation 22

C The Estimation Packet **22**

D Repository **23**
D.1 Layout 23
D.2 Commands 23

1 Abstract

[TODO: Abstract.]

2 Overview

2.1 Purpose of This Report

[TODO: Why write a self-evaluation at all? Who is the reader (you, an interviewer, the team)? What period of work does it cover, and as of what date and commit?]

2.2 The Team

[TODO: Who owned which subsystem? What did each contribute to the parts this report covers?]

2.3 Development Process

[TODO: How was the work actually done: what did you design and write yourself, what came from AI-assisted sessions, and how did you review and verify that work?]

2.4 How to Read This Report

[TODO: A short map of the parts. Write this last.]

3 Problem

3.1 Objective

[TODO: What must the robot do, in one paragraph? Which PRD rules matter most to vision: right-lane driving, the exact traffic-light rules (green to enter, yellow only once inside, never on red), the stop sign at the exit?] [TODO: Obstacle avoidance: The PRD's executive summary states it as a must. Was it formally downgraded? By whom, and when?]

3.2 Constraints

[TODO: Which constraints actually shaped your design decisions, and which were just boxes to tick? Consider: $100 BOM, the PCB requirement, 12 V limit, size (10 x 11 x 11 cm), local processing only, start button and 5 s countdown, elapsed time in tenths of a second.] [TODO: Battery display: the PRD asks the power system to alert navigation. Why was battery life moved to the display?]

3.3 The Course

[TODO: Describe the course the robot was built and tested on. Where does it differ from the PRD's course, and does each difference make the vision problem easier or harder?]

Table 1: **[TODO: caption]**

PRD	Team course
[TODO: rows]	

3.4 Requirements

[TODO: Turn the PRD's prose into requirements that can each be checked one way. For each: is it straight from the PRD, or derived (something vision needs so a PRD rule can be met)? What numbers will you hold yourself to, and why those numbers?] [TODO: The PRD's validation guidance asks for each object's detection field (range, angles, minimum size). Its own requirement, or part of the detection ones?]

Table 2: **[TODO: caption]**

ID	Requirement	Source
[TODO: rows]		

Part I
Design

4 Architecture

4.1 Structure of an Autonomous Robot

Navilott drives a street-style course without intervention: it follows a lane, stops at stop signs and stop lines, obeys traffic lights and avoids obstacles. Its software is divided into six functional areas, the usual chain from sensing to motion (Douglas, 2020b), and each boundary exists because the task needs it (Figure 1). The areas are defined by what each hands to the next:

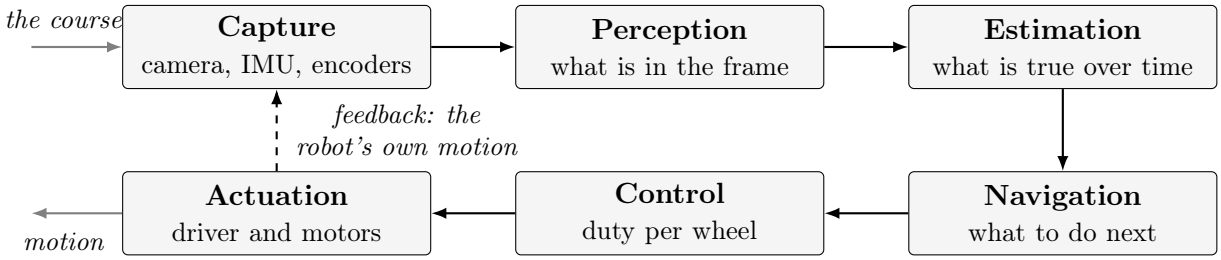


Figure 1: The six areas as a closed loop.

- **Capture** produces raw measurements: camera frames, gyro and accelerometer readings, and wheel encoder counts, each stamped with the time it was taken.
- **Perception** turns one frame into what that frame shows: lane position, a stop line, a stop sign, a traffic light's color, an obstacle.
- **Estimation** turns perception's results and the motion sensors into what the robot believes over time: its heading, the distance it has traveled, and whether a stop line is really ahead rather than seen once.
- **Navigation** turns that belief into a decision: keep the lane, stop, wait, or cross an intersection, and at what speed.
- **Control** turns the decision into a duty for each wheel, and during turns corrects it against the speed the encoders report.
- **Actuation** turns duties into motion: PWM signals switch the motor driver, which applies battery voltage to the motors.

Perception and estimation. The most consequential boundary is between these two areas. A single frame can be wrong: glare hides a lane line, another lane's stop line enters the view, a frame arrives late. Perception therefore reports only what each frame shows,

and estimation decides what to believe by combining frames with each other and with the sensors the camera cannot replace. The camera measures where the robot is relative to the lane; the gyro and encoders measure how it is moving.

Two feedback paths. What actuation does is measured and returned, so each decision is based on what the robot actually did. The loop closes along two paths:

- **Within control**: the encoders report each wheel's speed every frame. During lane keeping, control learns from it how much speed each unit of duty produces; during a turn, it corrects each wheel's duty against the commanded speed, so the turn keeps its shape whatever the battery voltage or the floor.
- **Across frames**, through the camera and estimation: the next frame shows where the last command put the robot in the lane, and the gyro and encoders show how far it turned and traveled, so the next decision corrects any error the last one left.

Keeping this loop running at the camera's frame rate is the constraint behind every later decision.

4.2 Single-Board Design

The first decision is where each functional area runs, and in particular how perception on the camera side connects to actuation at the motors. Four criteria decided it: the control rate the robot actually needs, a bill of materials within the $100 budget, processing done on the robot rather than off-board, and the development time a second processor would add.

Three options were considered (Figure 2):

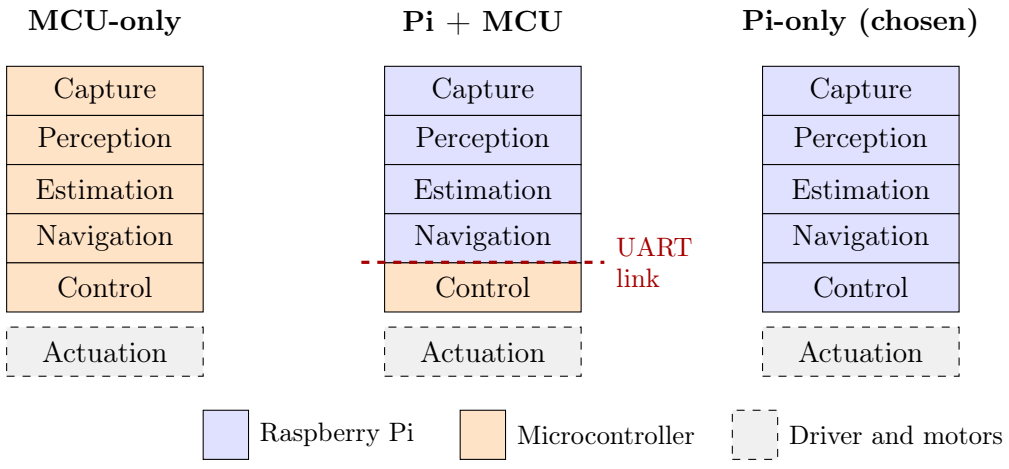


Figure 2: Where each area runs under the three options.

- **MCU-only**: an STM32 or Arduino runs perception and drives the motors.
- **Pi + MCU**: the Pi runs perception and sends a packet over UART to an STM32 or Arduino, which runs a fast PID loop and generates the PWM.
- **Pi-only**: the Pi runs perception and drives the motor driver directly with PWM.

MCU-only: feasible, but against the criteria. Microcontrollers can run camera-based vision: parts such as the STM32H7 and ESP32-S3 accept a parallel digital video port (DVP) camera and have enough memory, internal or external, for small frames. The option was rejected for what it would cost this design:

First, the camera: the robot needs a wide field of view to see the lane lines and the traffic light from a stop line (Section 5), and the wide-angle sensors considered (IMX219, IMX290) are raw Bayer sensors on MIPI CSI-2 with no on-sensor image processing. They need a host with a CSI-2 receiver and an ISP, which most microcontrollers in the budget lack; a DVP camera would avoid that but would also take roughly a third of the I/O the motors, encoders and sensors need.

Second, the software: the pipeline undistorts a wide-angle lens, converts color spaces, finds contours and maps the image to the ground, all through OpenCV, and every phase is tested off the robot on recorded frames. On a microcontroller each of those would be reimplemented in C, without the replay and recording tools the design relies on.

A single 480×270 color frame is also 389 KB of raw pixel storage before any intermediate images, which fits only the largest microcontrollers or external PSRAM, at a cost in memory bandwidth.

Pi + MCU: more flexible, but with more overhead. In this layout the Pi runs perception and sends a command packet over UART to a microcontroller, which runs the wheel control loop, generates the PWM and reads the encoders (Appendix D gives the full specification and packet format). It has three real benefits:

- **Motor loop**: the motor loop runs at its own rate, independent of the frame rate and of Linux scheduling.
- **Encoders**: the encoders are decoded by the microcontroller's timer hardware rather than by software.
- **Motor safety**: the motor safety no longer depends on the Pi: a watchdog on the microcontroller brakes the motors if commands stop for any reason, including the Pi's process dying or the Pi losing power.

Its costs are a second processor's worth of design: a packet protocol with framing, checksums and a defined response to a lost link; two clocks to reconcile, since frames are stamped on the Pi and encoder and IMU samples on the microcontroller, where an error misaligns the two without any visible failure; and two codebases to build, test and keep in step.

Decision: Pi-only. The Pi Zero 2 W runs every area. Choosing it means giving up the three benefits of the microcontroller, and the design was judged against each.

First, a faster motor loop, buys little here: steering corrections come from the lane offset, which changes once per frame, so the camera sets the control rate in either layout, and a loop far faster than 20 Hz would mostly repeat the last command.

Second, encoders decoded in hardware, is largely provided by the pigpio daemon, which samples the pins in native code and queues every edge; a late Python thread delays the

counts but does not lose them. The same daemon drives the Pi's two hardware PWM channels (GPIO12 and GPIO13), so the waveform holds steady when the software is late, and scheduling delay postpones a change of command without distorting the signal.

Third, motor supervision independent of the Pi's software, is given up, and the conditions under which that is acceptable are set out below.

In return, the robot has one board, one clock and one codebase, which every phase can be tested against off the robot. The cost is that timing and safety now depend on one Linux system: control shares the processor with perception, and stopping the motors depends on the Pi's software. Sections 4.5 and 4.6 describe how the design limits those costs. That 20 Hz is sufficient is an assumption of the design, not a result of it; Section 12 tests it on the course.

The residual hazard and why it is accepted. Without a second processor, every software path that can stop the motors runs inside the robot's Python process. If that process is killed, or hangs while holding the interpreter lock, neither the watchdog nor the run loop can act, and the pigpio daemon continues to generate PWM at the last commanded duty: the robot keeps driving on its last command. A reset or loss of power on the Pi does not carry this hazard, because the motor driver's standby pin returns to its default low state and disables the outputs; the hazard is specific to a failed process on a running Pi.

The design accepts this under one criterion: a runaway robot must be unable to harm anyone and must always be stoppable by hand. The robot meets it on the course because it is small and slow (1Kg, top speed about 4 cm/s at the base duty), its N20 motors stall against light resistance, the course is enclosed and kept clear of people, and an operator is present for every run, since no run can start without the start button. The physical intervention is to lift the robot, which takes the wheels off the floor, and to open the power switch, which removes the motors' supply.

A hardware-independent disable was considered: a retriggerable timer on the driver's standby pin, held enabled only while the software toggles a GPIO every frame, would cut the motors whenever the process stopped, whatever its state. It was not adopted because the criterion above was already met for the course, and it is the first change recommended for any use outside the course (Section **??**).

4.3 Additional Hardware

Selecting the Pi as the central computer fixes where processing happens, but the robot still needs sensing beyond the camera, a way to drive its motors, regulated power, and an interface for the operator. Table 3 lists each component, how it connects to the Pi, and which functional area it serves; the paragraphs that follow cover only the choices that shaped the design.

Table 3: Hardware around the Pi.

Part	Role	Interface	Area
Innomaker CAM-MIPI290RAW (IMX290)	Wide-angle camera	CSI-2	Capture
MPU-6050	Gyro and accelerometer	I^2C (0x68)	Estimation
N20 encoders (x2)	Wheel rotation, quadrature	GPIO, 2 pins each	Estimation, Control
TB6612FNG	Dual H-bridge motor driver	GPIO: PWM + direction	Actuation
N20 gear motors (x2)	Drive wheels	TB6612 outputs	Actuation
ADS1115	Battery voltage (via 4:1 divider)	I^2C (0x48)	Monitoring
TM1637	4-digit display	GPIO, 2 pins	Interface
Push button	Start	GPIO	Interface
MP1584EN buck converter	Battery to 5 V for the Pi	Power	Power
3S 18650 pack	12.6 V full, 11.1 V nominal	Power	Power

Sensing. The camera was the one component replaced during development, and the change moved work from geometry to software. The original Raspberry Pi Camera V2 (IMX219) cropped its field of view in the sensor mode the Pi selected, leaving a 22 cm blind spot ahead of the robot: stopped at a stop line, it could not see the lane lines and the traffic light in the same frame, so the stopping position traded one against the other. The IMX290-based replacement uses the same CSI-2 interface and reduces the blind spot to 10 cm, so a single stopping position serves both. Its wider lens adds barrel distortion, which perception removes on every frame using a per-lens calibration; this adds a fixed processing step and ties the calibration to the lens and its mounting (Section 5). The camera alone measures where the robot is relative to the lane, while the IMU and wheel encoders supply heading and wheel motion to estimation (Section 7).

Motion. The Pi's GPIO pins supply only a few milliamps, far below what the N20 motors draw, so a TB6612FNG H-bridge switches battery power to them under PWM from the Pi. The driver can also short-brake a motor (connect its terminals); the software uses this for every stop.

Power. One 3S pack powers everything along two paths: directly to the motor driver, and through a buck converter to the Pi's 5 V supply, from which the Pi supplies 3.3 V to the logic-level parts. Motor current lowers the pack voltage under load, and the buck converter keeps the Pi's supply steady as long as the pack stays above its minimum input. The Pi has no analog input, so an ADS1115 reads the pack voltage through a 4:1 divider.

Battery protection and its limits. The software's voltage thresholds protect the pack against over-discharge during a run: it warns at 10.5 V and ends the run after three consecutive readings at or below 9.9 V, an average of 3.3 V per cell. The reading is smoothed, so a brief sag under motor load does not end a run, but a sustained heavy load reads lower than the resting voltage and can end it early. The thresholds cannot guarantee more than that. They see only the pack's total, so one weak cell can fall below a safe voltage while the average stays above the limit; they act only while the robot's software runs, while the Pi and buck converter continue to draw from the pack when it does not; and they offer no protection against over-current or a short circuit. The cells carry their own protection circuits, which covers those cases. Charging is done off the robot with a balance charger, which also restores equal cell voltages between runs.

Interface. A start button holds the robot until the operator is ready: no motor command is sent before it is pressed. A TM1637 display shows the robot's state and the run time, and the final time when the run ends.

Integration. Every component except the camera connects through a PCB that carries the power rails and signal routing, with soldered headers for each external part. The chassis holds what the board cannot: the motors, the camera at a fixed height and angle, the display, the button and the battery. Fixing the camera mount matters beyond packaging: the lens calibration and the stop-line distance table are valid only for one mounting position, so moving the camera means repeating both.

4.4 Software Language

With the hardware interfaces established, the next decision is the language that turns sensor data into motor commands. Four criteria decided it: access to a mature computer-vision library, a path from the Pi's camera stack into that library, fast iteration on the robot, and code that can be tested off the robot. Python met all four:

- **OpenCV**: the computer-vision library, written in C++, with Python bindings for every algorithm the pipeline uses.
- **GStreamer**: OpenCV opens the camera through a GStreamer pipeline built on libcamera, so frames arrive from the Pi's camera stack directly as OpenCV images.
- **Iteration**: no compile step; a change is edited on the Pi and run immediately, which suits tuning thresholds on the course.
- **Testing**: replacing the camera, sensors and motors with test doubles is direct in Python, so every phase is tested on a laptop with synthetic or recorded frames.

The cost: an interpreter and a global lock. Python is interpreted, so each Python-level operation costs far more than its equivalent in C. It also has a Global Interpreter Lock (GIL): only one thread executes Python code at a time, whichever core it is on. Native code

called from Python may release the lock while it works, but only where the library does so explicitly, so the cost of the GIL depends on which calls a thread spends its time in:

- **OpenCV**: the generated Python bindings release the lock for the duration of each wrapped C++ function, including `VideoCapture.read()`, which waits for the next frame without it. Converting NumPy arguments to and from OpenCV images happens with the lock held, but contiguous arrays are not copied.
- **GStreamer and libcamera**: run on their own native threads and never take the lock.
- **NumPy**: releases the lock for many whole-array operations on numeric data, but not for indexing, small arrays, or work that loops in Python.
- **Python code**: everything between those calls (iterating over contours, applying gates, building packets, evaluating navigation rules) holds the lock throughout.

Why the cost is acceptable. The pipeline keeps per-pixel work inside OpenCV calls and leaves Python to connect them and handle their results. Rewriting estimation and navigation in C++ would save under 1% of the frame (Section 12). The perception stage times include the Python that drives their OpenCV calls, such as geometry's loop over candidate contours, and that share was not measured separately; a C++ port could shorten perception by that share, but no more, since the OpenCV work is already native. The conclusion is limited to this workload: a pipeline with more Python per pixel would weigh the choice differently. The GIL does constrain the threads that run alongside the main loop, which the next section describes.

4.5 Background Threads

Python and its native libraries define the implementation environment, but not how sensing, processing and motor supervision run at the same time. Navilott runs as one Python process plus the pigpio daemon (Figure 3). The main loop does the per-frame work; every other path either collects data at its own rate or supervises the main loop, and each device has exactly one owner, so no two paths contend for the same hardware. Every frame and reading is stamped on the same monotonic clock, so data from different paths line up without a synchronization step. Table 4 gives each path's rate and what it does when the main loop falls behind.

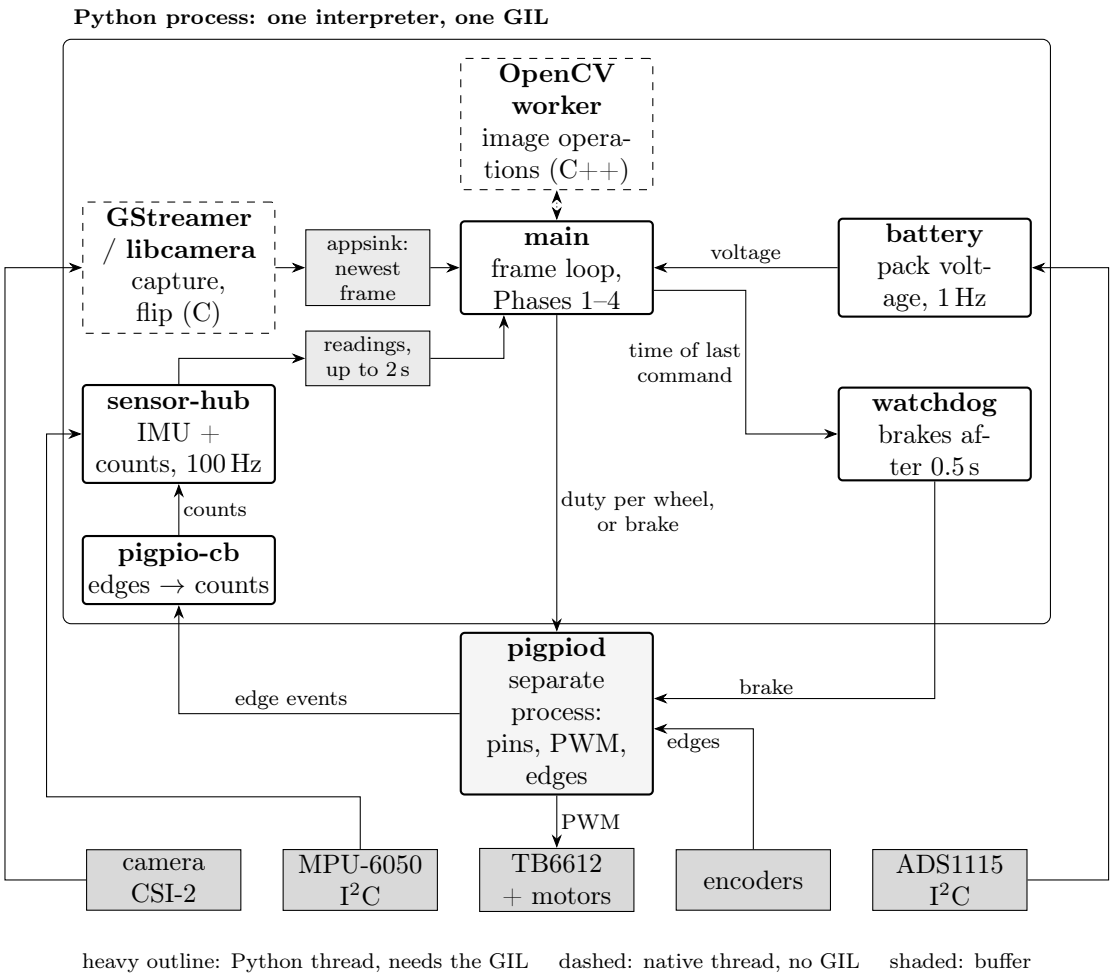


Figure 3: Execution model: threads of the Python process, the pigpio daemon, the buffers between them, and the hardware each one owns.

Table 4: Execution paths, their rates, and their behavior when the main loop is late.

Path	Rate	When the main loop is late
Main loop	Per frame (20 Hz)	Sends its next command late; the motors hold the previous duty
GStreamer/libcamera	20 Hz	Keeps only the newest frame; older frames are dropped
Sensor hub	100 Hz	Buffers stamped IMU readings, up to 2 s; beyond that the oldest are dropped and counted
Encoder callbacks	Per edge	Accumulate counts; nothing is lost however late the read
Battery monitor	1 Hz	Keeps a smoothed voltage; only the latest value matters
Motor watchdog	Continuous	Brakes the motors after 0.5 s without a command
OpenCV worker	On demand	Not affected; works only inside the main loop's calls
pigpiod	Continuous	Keeps generating PWM at the last duty and timestamping edges

Three freshness policies. The paths that feed the main loop keep their data in three different ways, because each kind of data loses its value differently with age:

- **Frames are newest-only.** A frame describes the scene at one instant, and an old one is worse than none, so the appsink keeps a single frame. A late loop processes the current scene and never works through a backlog; the frames it missed are simply lost.
- **IMU readings are a bounded history.** Heading is the integral of yaw rate, so every reading between frames contributes, and the hub keeps them all for the next frame. The buffer holds 2 s; if the loop stalls longer, the oldest readings are dropped and the frame reports how many. Estimation further clamps the time step it integrates over to 0.5 s, so after a long stall the heading under-counts any turn made during it; lane vision, or the gyro target of a turn in progress, must then recover the error.
- **Encoder counts are stateful.** The callbacks add each edge to a running total, and each frame reads the total afresh, so distance is exact however late the read. Only the speed derived from it degrades, since it becomes an average over a longer window.

The GIL and the sensor hub. Of the background paths, only the sensor hub runs Python on a deadline: one reading every 10 ms. When the main loop is inside an OpenCV call, which releases the lock, the hub runs immediately; when the main loop is running Python, the hub waits for the interpreter to switch threads. Python requests a switch after 5 ms, but the hand-over happens only when the running thread reaches a point where it can release the lock, so a reading can be delayed by a sizeable fraction of its period. Two measures limit the effect. The hub holds the lock as briefly as possible: the IMU's 14-byte sample block is read in one I^2C transfer at 400 kHz, so most of each reading is spent in the bus driver. And each reading is stamped when it is taken, so a delay changes when the reading is processed but not the time it is integrated at. Python's garbage collector, which

can pause every thread at an unpredictable moment, is disabled during a run and collects only when the robot is braked or every 200 frames.

4.6 Linux Scheduler

The execution model above depends on each thread getting a core when it needs one, and on Linux that is the scheduler's decision. The engineering contract follows from what the scheduler does and does not promise. The Pi runs kernel 6.18.50 (`rpt-rpi-v8`) with a preemptible configuration (`PREEMPT`), whose default policy is Earliest Eligible Virtual Deadline First (EEVDF). It shares CPU time among runnable tasks and preempts them at will, but it is not a hard real-time scheduler: it gives ordinary threads no bound on when they will run. Navilott does not take control of the scheduler (no core pinning, no real-time priorities). The design therefore treats every path as one that can be late, tolerates lateness wherever it can, and limits what lateness can cause where it cannot (Table 5).

Table 5: Timing failure modes, their mitigations, and what remains.

Failure mode	**Mitigation**	**Residual limitation**
A frame takes longer than 50 ms	Newest-frame appsink: the next frame is current, not queued; readings are buffered	The command arrives late; the motors hold the previous duty until it does
A sensor reading is late (scheduler or GIL)	Readings stamped when taken; buffered for 2 s	A reading delayed past its 10 ms slot is taken late, though recorded at its true time
The frame loop stalls (a long frame, a camera that stops delivering)	The watchdog short-brakes the motors after 0.5 s without a command; Phase 3 clamps its time step to 0.5 s	Heading under-counts any turn made during a stall longer than 0.5 s; a camera that stops delivering leaves the robot braked until stopped by hand
The Python code raises an exception	The run loop brakes, then puts the motor driver in standby, before anything else	None, provided the interpreter is still running
The Python process is killed, or the interpreter hangs while holding the GIL	None in software: the watchdog is a thread of the same process	**Unresolved.** pigpiod keeps generating PWM at the last duty; the motors run until the robot is stopped by hand (Section 4.2)
The battery runs low	Smoothed voltage; warning at 10.5 V, run ended after three readings at or below 9.9 V	The voltage is read under load, which sits below the resting value, so a heavy load can end a run before the pack is empty
OpenCV workers compete with other threads	OpenCV limited to two threads, leaving the remaining cores to GStreamer, the sensor hub and pigpiod	A different image size or workload may favour a different limit

How much lateness each path tolerates. The contract can be stated per path, as the delay each one absorbs without any effect on the robot's behaviour:

- **Sensor readings**: a late reading is still integrated at its true time, so delays are absorbed up to the 2 s buffer, and heading stays exact across frame gaps of up to 0.5 s.
- **Frames**: a late frame delays one command but creates no backlog, so lateness never accumulates from one frame to the next.
- **Commands**: the motors hold the previous duty, and the watchdog intervenes only after 0.5 s, ten frame periods, without a new command.

These are design bounds, not measured ones. The design additionally expects ordinary scheduling delay to be far shorter than any of them, since the Pi has four cores and fewer busy threads than cores for most of each frame. That is an expectation, not a property of the scheduler: it holds only as far as it has been measured, and Section 12 reports the measured wake-up latency distribution and the worst delay observed. Real-time priorities would narrow the delay, at the risk of a runaway thread starving the rest of the system, and are left as an option if the measured delays approach the bounds above.

Three different stops, one unresolved. The table separates three conditions that are easy to treat as one. When the *command stream* stops but the process runs, the watchdog brakes the motors. When the *application* stops through an exception, the run loop brakes them on its way out. When the *process* itself stops, or the interpreter hangs, no software on the Pi stops the motors: pigpiod outlives the process and keeps the last PWM duty. The first two are handled in software. The third is an unresolved safety limitation of the single-board design, accepted under the conditions in Section 4.2 and covered only by stopping the robot by hand.

With the runtime's timing limits and their mitigations established, the remaining concern is making the environment it runs in the same on every Pi and every boot.

4.7 Software Architecture

[TODO: After deciding on the main architecture and hardware peripherals to use, how was the software architecture designed? What were some of the key decisions made and why? How did the software evolve over time and what changes, if any, were made to the architecture?]

5 Capture

5.1 Camera Path

[TODO: How does light become a frame in Python? Which stages does the frame pass through, and which of them did you configure?]

5.2 Design Decisions

[TODO: Which capture decisions mattered, and what would have gone wrong without each? Consider the pinned sensor mode, the output size, keeping only the newest frame, flipping in GStreamer, and who assigns frame ids and timestamps.]

5.3 Failure Handling

[TODO: What happens when a frame is dropped, and when the camera dies? Which requirement does this serve, and how do you know it works?]

5.4 Open Questions

[TODO: What about capture is still unmeasured or uncertain? For example, the delay from exposure to timestamp, or exposure settings under course lighting.]

6 Perception

6.1 Approach

[TODO: Why classical computer vision rather than a learned model? What about this course made that a reasonable bet, and what did you give up?]

6.2 Stage Order

[TODO: What are the stages, in order, and why that order? Why is there no memory between frames at this phase?]

6.3 Lane Boundaries and Offset

[TODO: How are lane lines found, and how does a set of lines become one steering error? What does the sign of the offset mean? What happens when only one line is visible?]

6.4 Stop Lines

[TODO: How is a stop line told apart from a lane line? How is its distance measured, and in what units?]

6.5 Traffic Lights

[TODO: What made traffic lights hard on this course? How did the approach change once you found the lights were bare LEDs (glow mode)?]

6.6 Stop Signs

[TODO: How are stop signs detected, and how well tuned is the detector on real course frames?]

6.7 Debugging Perception

[TODO: How do you see why a frame failed? How do you see why a gate failed? Why would it be important to see this at all?]

7 Estimation

7.1 Role

[TODO: Where is the line between perception and estimation, and why draw it there?]

7.2 Lane Filter

[TODO: How is a raw offset smoothed, and when is a frame not trusted? What do vision, hold and stale mean, and why those thresholds?]

7.3 Heading Tracker

[TODO: What does the IMU add while vision is lost? What does heading actually measure, and what are its limits?]

7.4 Voting

[TODO: Why vote across frames for traffic lights, stop signs and stop lines? Why a window of three?]

7.5 The Packet

[TODO: What does navigation receive each frame? Which fields matter most? (Full field list in Appendix C.)]

8 Navigation

8.1 Scope and Attribution

[TODO: What is the role of navigation, and how was the design chosen? What was considered and what were the tradeoffs?]

8.2 The Contract

[TODO: What is the interface between vision and navigation? What must a navigator always do, and what must it never do? Why fix this before navigation existed?]

8.3 Driving Rules

[TODO: How does the robot respond to each light state, stop signs, stop lines and a lost lane? Do these rules match the PRD?]

8.4 Measured Robot Response

[TODO: How the robot actually responds to commands? How does that compare to the contract?]

8.5 Intersections and Route

[TODO: How is a turn sequenced, and how does the robot know which way to go at each intersection?]

Part II
Evidence

9 Testing

9.1 Scope

[TODO: What does testing check that verification doesn't, and the reverse? Where is the line between the two in this report?]

9.2 Testing Without the Robot

[TODO: How can the pipeline run with no robot attached? Which design choices made that possible, and when were they made?]

9.3 The Suite

[TODO: How large is the suite, what does it cover, and how long does it take to run? Which tests need the robot, and how are they selected?]

9.4 What Testing Proves

[TODO: Which requirements can software tests actually verify, and which can they not? Where did tests catch real bugs? Where did a passing suite hide a problem that only showed on the course?]

10 Diagnostics

10.1 Why Instrument the Pi

[TODO: What can go wrong on the Pi that the pipeline's own logs can't show? Why did that become worth building tools for?]

10.2 What Is Recorded

[TODO: Which recorders exist, and what question does each answer? Which ones actually changed a decision?]

Table 6: [TODO: caption]

Recorder	Question it answers
[TODO: rows]	

10.3 Observer Effect

[TODO: How do you know the recorder doesn't change the run it measures?]

10.4 What It Showed

[TODO: What did diagnostics reveal about frame rate, load, throttling or scheduling? Should the numbers go here, or only in Results?]

11 Verification

11.1 What Counts as Verified

[TODO: What has to be true before you call a requirement verified? Which hardware does it have to pass on, and do older results (IMX219) count?]

11.2 Routines

[TODO: What is a routine, and why run hardware checks this way instead of ad hoc? What does a request card fix in advance, and what does each run record?]

11.3 Checks by Requirement

[TODO: For each requirement: one check, with its pass criteria. Which requirements have no check defined yet?]

Table 7: **[TODO: caption]**

ID	Check and pass criteria
[TODO: rows]	

11.4 Whole-Robot Checks

[TODO: Which routines test the robot as a whole rather than a single requirement (figure-eight, course run, power profile ...)? What do they add?]

12 Results

12.1 Status by Requirement

[TODO: What grade does each requirement get, and on what evidence? Decide your grading scale first (e.g. Pass / Fail / Partial / Open) and define it in the caption.]

Table 8: **[TODO: caption, with the grading scale]**

ID	Grade	Evidence
[TODO: rows]		

12.2 Routine Results

[TODO: One row per routine run: routine, date, commit, trials, verdict, key number.]

12.3 Course Runs

[TODO: What happened when the robot ran the course? Early runs drifted right and failed at intersections. Has the stop-line fix resolved that?]

12.4 Measured Robot Response

[TODO: If the maneuver numbers live here rather than in Navigation, put them here.]

Part III
Assessment

13 Limitations

13.1 Requirements Not Met

[TODO: Which requirements fail, and what does each failure mean for how the robot drives?]

13.2 Requirements Not Yet Shown

[TODO: Which requirements have no hardware evidence either way? What is blocking each?]

13.3 Departures from the PRD

[TODO: Where does the robot's behavior differ from what the PRD asks?]

13.4 Limits of the Design

[TODO: What can this design never do well, however well it's tuned?]

14 Discussion

14.1 What Worked

[TODO: Which decisions would you make again, and what evidence says they were right?]

14.2 What Didn't

[TODO: Which decisions cost you, and when did you realize it?]

14.3 What I Would Do Differently

[TODO: With what you know now, what would you change in the design, the process, or how the team worked? How would you use AI-assisted development next time?]

14.4 What Remains

[TODO: What has to happen before Design Day, in priority order, and why that order?]

15 Conclusion

[TODO: If a reader only reads this section, what three things should they take away? What the pipeline is; how it measured against its requirements; what matters most next.]

References

G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.

Brian Douglas. Autonomous navigation. MATLAB Tech Talks video series, MathWorks, 2020a. URL `https://www.mathworks.com/videos/series/autonomous-navigation.html`. Six parts. Accessed 2026-10-09.

Brian Douglas. Autonomous navigation, part 1: What is autonomous navigation? MATLAB Tech Talks, MathWorks, 2020b. URL `https://www.mathworks.com/videos/autonomous-navigation-part-1-what-is-autonomous-navigation-1592993748308.html`. Video, 11:29. Accessed 2026-10-09.

Mike Orduna, Ignacio Rubio, and Ana Carlos. Navilott Vision. `https://github.com/Simbyote/Navilott_Vision`, 2026.

Texas State University. AutoBots product requirements document. Technical report, Ingram School of Engineering, Texas State University, January 2026. Sponsored by Jeff Stevens and Mark Welker. Version 0.5.

Part IV
Supplements

A Hardware

[TODO: Parts list and pin map, for reference only. The reasons behind each part belong in Architecture.]

Table 9: **[TODO: caption]**

Part	Role	Interface
[TODO: rows]		

Table 10: **[TODO: caption]**

BCM	Header	Use
[TODO: rows]		

B Parameters

[TODO: Which tuning values does a reader need on record to reproduce your results? Group by phase.]

B.1 Capture

[TODO: Sensor mode, output size, frame rate.]

B.2 Perception

[TODO: ROIs, and the gates that matter.]

B.3 Estimation

[TODO: Filter, hold, vote and confidence settings.]

C The Estimation Packet

[TODO: Every field navigation receives per frame: name, type, meaning, units and sign convention.]

Table 11: **[TODO: caption]**

Field	Type	Meaning
[TODO: rows]		

D Repository

[TODO: Where the code is, how it is laid out, and the commands a reader needs to reproduce a result.]

D.1 Layout

[TODO: One line per directory under `vision_stack/src/`.]

D.2 Commands

[TODO: The few commands that matter: setup, every-boot session, run, test, routines.]